

COGNITEK

AI-Powered Academic Assistant for Students

Project Technical Summary & Working Report

Prepared by Hans

1. What Is CogniTek?

CogniTek is a smart academic assistant application designed specifically for engineering students, particularly those studying under the Kerala Technological University (KTU) curriculum. Think of it as a personal student helper that listens to your lectures or class recordings and automatically organises the information for you — turning spoken words into tasks, study notes, exam reminders, and even answering your academic questions.

In simple terms: a student records or uploads a lecture audio. CogniTek listens to it, understands what was said, and automatically creates a to-do list, study flashcards, and exam alerts — all without the student having to type a single word.

2. Why Was CogniTek Built?

The core problem CogniTek solves:

- Students often miss important deadlines and assignments mentioned verbally in class.
- Taking manual notes during lectures is slow and often incomplete.
- Organising subjects, exams, and tasks separately in notebooks or multiple apps is inefficient.
- Students lack a single, intelligent tool that understands the KTU academic calendar and course codes.

CogniTek addresses all of the above by listening to lectures and doing the organisation automatically.

3. Where Is It Used?

- B.Tech students at KTU-affiliated colleges (primary target audience).
- Can be expanded to any university or academic institution with minimal changes.
- Accessible via a mobile browser on Android or as an installable mobile application.
- Works fully online — students only need an internet connection to use it.

4. Project Timeline

Phase	Description	Approx. Period
Phase 1 – Concept & Setup	Defined the problem, chose technology, set up the project structure and local server.	Early Development
Phase 2 – Core AI Pipeline	Built the audio upload → transcription → AI analysis → database save pipeline. Local Whisper model used.	Development
Phase 3 – Frontend Interface	Built the full mobile-friendly web interface: login, dashboard, recording page, flashcards, schedule, and chatbot.	Development
Phase 4 – Cloud Migration	Moved speech-to-text to Groq cloud API (Whisper Large v3). Removed local GPU dependency completely.	Mid Development
Phase 5 – User Accounts	Added Supabase authentication (login, register, password reset). Each student's data is now stored separately.	Mid-Late Development
Phase 6 – Exam & Placement	Added exam schedule tracking and placement milestone features. Timetable image scanning added.	Late Development
Phase 7 – Cloud Deployment	Backend deployed to Render.com (cloud server). Frontend deployed to Vercel. App is now publicly accessible.	Final Stage
Phase 8 – Android App Build	Packaged the web app into an Android APK using Capacitor. Ready for Play Store submission.	Final Stage

5. How CogniTek Works — Step by Step

5.1 Student Records or Uploads Audio

The student opens the CogniTek app and either records live audio (e.g., a class being conducted) or uploads a saved lecture recording. This audio file is sent to the CogniTek server over the internet.

5.2 Speech-to-Text Conversion (Transcription)

The audio file is sent to a cloud speech recognition service called Groq Whisper (powered by OpenAI's Whisper Large v3 model). This service converts everything spoken in the audio into plain written text. This is similar to how voice assistants like Google Assistant or Siri convert speech to text — but Whisper is specifically trained to handle technical, academic language accurately.

5.3 AI Reads and Understands the Text

The written text is passed to Google Gemini 2.5 Flash — an advanced AI language model. The AI reads the text carefully with a specific set of instructions that tell it to look for:

- Tasks and assignments mentioned (e.g., 'submit your report by Friday')
- Exam or class test dates (e.g., 'class test next Monday at 2 PM')
- Definitions, laws, or concepts worth turning into study flashcards
- Placement or internship drives mentioned (e.g., 'Infosys off-campus next week')

The AI also knows all KTU subject names and course codes, so it can correctly label tasks under the right subject (e.g., CST201 – Discrete Computational Structures).

5.4 Data Is Organised and Saved

The AI returns all the extracted information in a structured format. The server then saves this data into the database — specifically linked to the logged-in student's account. Nothing is shared between students.

5.5 Student Views Results on the App

The student can now see their tasks, upcoming exams, flashcards, and placement reminders — all automatically generated. They can also chat with the AI assistant called Sylens for academic help.

6. Key Features of CogniTek

Feature	What It Does
Voice Recording & Upload	Student records live or uploads saved lecture audio directly from the app.
Automatic Task Extraction	AI reads the lecture and creates a to-do list of assignments and deadlines automatically.
Flashcard Generation	Converts definitions, laws, and concepts from lectures into study question-answer cards.
Exam Schedule Tracking	Detects class test dates mentioned in audio and adds them to the student's exam calendar.
Placement Milestone Tracker	Tracks internship and job drive dates mentioned by professors or placement coordinators.
Sylens AI Chatbot	A smart assistant the student can chat with for help understanding tasks or academic topics.
Timetable Scanner	Student photographs their printed timetable — app reads and digitalises it automatically.
Individual User Accounts	Every student logs in separately; their data is private and not visible to others.
Mobile App (Android)	Packaged as an installable Android application — works like a native phone app.

7. Technology Stack

Below is a breakdown of every technology used in the project

7.1 Frontend (The App Screen the Student Sees)

Technology	Role / Purpose
React 19	The JavaScript framework used to build the interactive app interface (all screens and buttons).
Vite 7	A build tool that compiles and optimises the app code for fast loading.
Tailwind CSS v4	A styling library used to design the visual appearance of the app (colours, layouts, animations).
React Router v7	Handles navigation between different pages/screens within the app.
Axios	Sends requests between the app screen and the server (like a courier between the two).
Supabase JS	Connects to the authentication (login/signup) service from the app side.
Capacitor	Converts the web app into a native Android application (APK) for phone installation.
Lucide React	Icon library providing visual symbols used throughout the UI.

7.2 Backend (The Server — The 'Brain')

Technology	Role / Purpose
Python 3.10+	The programming language the server is written in.
FastAPI	A modern Python web framework that handles all incoming requests from the app.
Uvicorn	The program that actually runs (serves) the FastAPI server.
SQLAlchemy ORM	Manages the database — creates tables, saves, updates, and retrieves data.
Pydantic	Validates and checks incoming data to ensure it is in the correct format.
python-dotenv	Reads secret configuration values (like API keys) from a secure file.
psycopg2-binary	The connector that allows Python to communicate with a PostgreSQL database.

7.3 AI Models

Model	Provider	What It Does
Whisper Large v3	OpenAI / Groq	Converts lecture audio to text (speech-to-text transcription). Highly accurate even with technical terminology.
Gemini 2.5 Flash	Google DeepMind	Reads transcribed text and extracts tasks, flashcards, exam dates, and placement info. Also powers the Sylens chatbot.
Llama 3 (8B)	Meta / Groq	A lightweight, faster AI model used for the quick-response version of the Sylens chatbot (chat-fast endpoint).

8. Cloud Platforms Used

Platform	Purpose
Render.com	Hosts the Python backend server 24/7 on the internet. Free-tier cloud hosting.
Vercel	Hosts the frontend React app and makes it accessible via a public web link.
Supabase	Provides student login/registration (authentication) and also stores user profile data in the cloud.
Groq Cloud API	Provides fast speech-to-text conversion (Whisper) and fast AI chat (Llama 3) via internet API calls.
Google AI Studio	Provides access to Google's Gemini 2.5 Flash AI model for intelligent text analysis.
GitHub	Version control — stores all code safely and enables team collaboration.
ngrok (dev)	Used during development only — created a temporary public internet link to the local server for mobile testing.

9. Database Design

CogniTek uses a relational database (like a structured spreadsheet with linked tables) to store all student data. In production (live deployment), it uses PostgreSQL — an industry-standard database. During local development and testing, a lightweight SQLite database was used.

Table Name	What It Stores
tasks	All assignments and to-do items extracted from lectures for each student.
flashcards	Study question-and-answer card decks generated from lecture content.
exam_sessions	Exam and class test schedules extracted from audio or entered manually.
placement_milestones	Internship and placement drive deadlines and company information.

10. Major Changes During Development

The project evolved significantly from its initial design. Key changes are listed below:

10.1 Speech Recognition: Local → Cloud

Initially, the speech-to-text conversion was done using a local AI model (OpenAI Whisper) running directly on the developer's computer GPU. This required special hardware (NVIDIA GPU), Python version 3.10 specifically, and a large model file downloaded locally.

CHANGE: This was replaced with Groq's cloud Whisper API. Now, audio is sent to Groq's servers over the internet, processed there, and the text is returned. This eliminated the hardware requirement entirely, making the server deployable on any basic cloud machine.

10.2 Database: Local File → Cloud PostgreSQL

Development used a simple local SQLite database file. For production (live), this was upgraded to a cloud-hosted PostgreSQL database via Supabase — ensuring data is safe, scalable, and accessible from anywhere.

10.3 Added User Authentication

Early versions had no login — all students shared one data pool. Supabase Authentication was integrated to provide individual student accounts (login, register, forgot password). Every piece of data is now tied to a unique student ID.

10.4 Dual AI Chat System

Initially there was one chatbot endpoint using Gemini. A second, faster endpoint was added using Groq's Llama 3 model for quicker responses in casual conversations. Gemini is used when higher quality or more complex responses are needed.

10.5 Android App Packaging

The app began as a web application only (accessible via browser). Using a tool called Capacitor, it was packaged into a native Android APK — making it installable on any Android phone exactly like a regular downloaded application.

10.6 Timetable Image Scanning

A new feature was added where students can photograph their printed college timetable. The image is sent to Gemini's vision capability, which reads and extracts the schedule automatically, digitalising the weekly timetable without manual entry.

10.7 Placement & Exam Tracking

Initially the app only tracked academic tasks and flashcards. Two new modules were added: an Exam Schedule tracker (for class tests) and a Placement Milestone tracker (for job/internship drives) — both automatically populated from lecture audio.

11. Application Screens (Pages)

Screen	Description
Login / Register	Student account creation and sign-in screen with email and password.
Forgot Password	Sends a password reset link to the student's email.
Onboarding	First-time setup where the student selects their branch, semester, and enrolled subjects.
Home (Dashboard)	Displays the student's pending tasks, priority indicators, and quick stats.
Record	Audio recording and upload interface — the main entry point for lecture processing.
Study (Flashcards)	Flip-card study interface for reviewing AI-generated question-answer decks.
Exam Mode	Displays upcoming class tests and exams; allows manual entry of exam details.
Schedule	Shows the student's weekly class timetable (digitalised from photo or manual entry).
Sylens (Chat)	AI chatbot page where students can ask academic questions or get task summaries.
Profile	User account settings, enrolled subjects, and data management options.
Research Mode	Advanced topic exploration feature for deeper academic study assistance.

12. System Architecture Overview

The system is structured as three main layers that communicate with each other:

STUDENT'S PHONE (Android App / Browser)

↕ Internet (HTTPS requests)

COGNITEK BACKEND SERVER (Render.com)

↕ API Calls (over Internet)

CLOUD AI & DATABASE SERVICES

- Groq API → Speech-to-Text (Whisper)
- Google Gemini API → AI Analysis & Chatbot
- Supabase → User Accounts & Database

The student's device never runs any AI locally — all processing happens on remote servers, making the app lightweight and battery-friendly.

13. Security & Privacy

- All communication between the app and server uses HTTPS (encrypted connection).
- Student passwords are never stored in plain text — Supabase handles secure authentication.
- Each student's data (tasks, flashcards, exams) is isolated by their unique User ID.
- API keys (for Gemini, Groq) are stored as environment variables on the server — never exposed to the user.
- Temporary audio files are deleted from the server immediately after processing.

14. Quick Reference Summary

Category	Details
Project Name	COGNITEK
Type	AI-Powered Academic Assistant Mobile & Web Application
Target Users	B.Tech Students (KTU, India)
Primary Language	Python (Backend), JavaScript/React (Frontend)
AI Models	Google Gemini 2.5 Flash, OpenAI Whisper Large v3, Meta Llama 3
Cloud Hosting	Render.com (Backend), Vercel (Frontend), Supabase (Database & Auth)
AI API Services	Groq Cloud API, Google AI Studio (Gemini)
Database	PostgreSQL (Production), SQLite (Development)
Mobile Platform	Android APK (via Capacitor)
Authentication	Supabase Auth (Email & Password)
Development Status	Completed — Ready for Presentation / Not for Play Store Submission
Backend Endpoints	15+ API endpoints covering all features
Frontend Screens	14 distinct application screens/pages